

Lab 04 — Building a Dynamic Array

DSA27 Lab Manual · Week 4 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 4

Contents

Part 0 — Before you start	2
Part 1 — Growing: <code>_resize</code> and <code>append</code>	4
Part 2 — Reading and writing: <code>__getitem__</code> and <code>__setitem__</code>	6
Part 3 — Inserting in the middle: <code>insert_at</code>	7
Part 4 — Removing: <code>pop</code>	9
Part 5 — The growth factor experiment	11
Part 6 — Measure it	13
Part 7 — Exercises at a glance	14
Part 8 — Take-home practice (not graded)	15
Part 9 — Bridge to Lecture 05: linked lists	15
Summary	16
Answers to the checkpoints	16

How to use this lab. From this week the lab is where you **build** the week's structure. For every part: read the Lecture 04 section it names, **draw before you code** (on paper, or with *viz.draw*), predict at each **Checkpoint** (answers at the end), then write one method and run the tests named for it before moving on. Build in the order given — each part stands on the one before. The worked solution is in `solutions/dsa/dynamic_array.py`, for after you have tried.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>dsa/dynamic_array.py</code> — <code>_resize</code> , <code>append</code> , <code>__getitem__</code> , <code>__setitem__</code> , <code>insert_at</code> , <code>pop</code>
Graded by	<code>tests/test_dynamic_array.py</code> (19 tests)

Connects to

Lecture 04 — Dynamic Arrays; used again by Stack (Week 6) and SlowQueue (Week 7)

What you will be able to do

1. explain the difference between a dynamic array's **size** and its **capacity**, and draw both;
 2. grow a fixed-size Array by the three steps *allocate, copy, switch*;
 3. write `append` so that it is amortised $O(1)$, and say why its worst case is still $O(n)$;
 4. translate negative indices and check bounds against the size, not the capacity;
 5. insert and remove in the middle by shifting — in the right direction;
 6. measure, with `resize_count` and with a timer, why growing by a **factor** beats growing by a **constant**;
 7. explain why `pop(0)` is $O(n)$, and what that will cost a queue in Week 7.
-

Part 0 — Before you start**0.1 Check your environment**

From the DSA27 folder, with the virtual environment active:

```
pytest -m "not challenge" -q           # the environment check: must pass
pytest tests/test_array.py -q          # the course Array you build on: given, must pass
pytest tests/test_dynamic_array.py -q # your exercise: 18 failed, 1 passed
```

The one test that passes before you write anything is `test_empty`: an empty `DynamicArray()` never calls a method of yours. Every other test calls one of your methods — most of them `append`, directly or through the constructor — and so hits `NotImplementedError`.

0.2 You have written half of this already

Week 2's `dsa/array_ops.py` had `insert_at(arr, size, index, value)` and `remove_at(arr, size, index)`: a **partly filled** Array plus a separate size saying how many slots are in use. A dynamic array is exactly that pair — the Array and its size — kept together inside one object, plus one new trick: when the Array is full, replace it with a bigger one.

If your Week 2 functions pass,

```
pytest tests/test_array_ops.py -k "insert_at or remove_at" -q # 9 tests
```

then the shifting loops of Parts 3 and 4 below are already in your hands. Open `dsa/array_ops.py` next to `dsa/dynamic_array.py`. (Lecture 04, "The contract with costs": `insert_at` and `pop(i)` are "Lecture 02's `insert_at` and `remove_at` ... moved inside the class".)

0.3 Read the skeleton

Open `dsa/dynamic_array.py`. The docstrings are the contract; the test file is the full specification.

Given — do not change	Yours
<code>make_block(capacity)</code> — returns <code>Array(capacity)</code>	<code>_resize(capacity)</code>
<code>__init__(values=(), growth=2)</code>	<code>append(value)</code>
<code>__len__</code> — returns <code>self._size</code>	<code>__getitem__(index)</code> , <code>__setitem__(index, value)</code>
<code>__iter__</code> — yields slots <code>0 .. size-1</code>	<code>insert_at(index, value)</code>
<code>__repr__</code> — <code>DynamicArray([...])</code>	<code>pop(index=-1)</code>

Read `__init__` closely — it tells you the four attributes you work with:

Attribute	Meaning
<code>self._block</code>	the <code>Array</code> underneath; <code>len(self._block)</code> is the capacity
<code>self._capacity</code>	how many slots <code>_block</code> has — keep it equal to <code>len(self._block)</code>
<code>self._size</code>	how many slots, from index 0, hold data — what <code>len()</code> returns
<code>self.growth</code>	the growth factor , 2 by default
<code>self.resize_count</code>	how many times <code>_resize</code> has run

Notice two things. First, a new array starts with **capacity 1**, not 0 — so the very first `append` always has room. Second, the constructor builds the array by calling `self.append` for each value, so until `append` works, even `DynamicArray([1, 2, 3])` fails.

0.4 The storage rule, this week

A `dsa/` structure stores its data only in the course `Array`, in node objects, or in another `dsa/` structure (Lecture 02). Here that means: **`_block` is an `Array`, and there is no Python list anywhere in the class** — not as the storage, not as scratch space inside `_resize`, not “just for a moment” in `insert_at`. You move every element yourself, one index at a time. That is how you see what each operation costs.

0.5 A picture to draw with

Throughout this lab you will want to see capacity and size at the same time. Put this helper in the scratch cell of notebooks/04-dynamic-array.ipynb (or a scratch file — **not** in `dsa/`):

```
from dsa.dynamic_array import DynamicArray
from viz.draw import draw_array
```

```
def show(arr):
    """Draw every slot of the block; the used ones (0 .. size-1) are marked."""
    slots = list(arr._block)          # peeking inside: for drawing only
    return draw_array(slots, done=range(len(arr)),
                      title=f"size {len(arr)}, capacity {arr._capacity}")
```

Reading `_block` from outside the class breaks encapsulation, which is fine in a notebook for learning and never fine in real code. The unused slots draw as `None`: they exist in the Array, but hold nothing the user put there.

Part 1 — Growing: `_resize` and `append`

1.1 The idea

Lecture 04, “Allocate, copy, switch”. A full Array cannot grow in place. So `_resize(capacity)`:

1. **allocates** a new block with `make_block(capacity)`;
2. **copies** the size elements across, index by index — $O(\text{size})$;
3. **switches** `self._block` to the new block, sets `self._capacity`, and adds one to `self.resize_count`.

The old block is no longer referred to by anything, so Python’s garbage collector reclaims it.

The lecture prints `_resize` in full because it is mechanical. The real exercise is `append`: *when* to call it, and *with what capacity*. The docstring tells you: when full, call `self._resize(self._capacity * self.growth)`.

1.2 Draw it

On paper, starting from `DynamicArray()` (capacity 1, size 0), draw the block after each of five appends of 'a' ... 'e'. For each resize, draw the old block, the new block, and an arrow for each copied element. Count the arrows.

Then check your drawing once Part 1 passes:

```
arr = DynamicArray()
for ch in 'abcde':
    arr.append(ch)
    show(arr)
```

Checkpoint 1. After the five appends above, give `len(arr)`, `arr._capacity`, `arr.resize_count`, `list(arr._block)`, and the total number of elements copied by all the resizes.

1.3 Write it

`_resize(self, capacity)` — the three steps of 1.1. Copy `range(self._size)`, not the capacity: only the used slots hold data.

`append(self, value)`, in words:

1. if the array is full (size equals capacity), resize to `capacity * growth`;
2. write value into slot size — the first free slot;
3. add one to size.

The order matters: **resize before you write**. Slot size of a full block does not exist.

1.4 Test it

```
pytest tests/test_dynamic_array.py -v -k "constructor or grows_beyond or doubling"
```

This selects three tests: `test_constructor_takes_an_iterable`, `test_grows_beyond_initial_capacity` (1,000 appends, every value survives every resize) and `test_doubling_keeps_resizes_logarithmic` (after 1,000 appends, `resize_count` must be more than 0 and fewer than 20). `test_append_and_index` also appends, but it reads with `arr[0]` — it waits for Part 2. The filter says `grows_beyond`, not just `grows`, on purpose: `-k grows` would also pick up `test_insert_at_grows_when_full`, which waits for Part 3.

1.5 When it fails

Writing before checking for room. Every test that appends twice fails with

```
IndexError: Array index 1 out of range for length 1 (valid: 0..0; no negative indices)
```

The message comes from the course `Array`, not from your class: the second append wrote to slot 1 of a one-slot block, *then* noticed it was full. Move the “if full, resize” to the top of `append`.

The same message appears if `_resize` builds and fills the new block but **forgets to switch** `self._block` to it — the writes still go to the old, one-slot block. Read `_resize` line by line: allocate, copy, switch.

Forgetting to update `self._capacity` in `_resize`.

```
IndexError: Array index 2 out of range for length 2 (valid: 0..1; no negative indices)
```

The first resize makes a block of 2, but `_capacity` still says 1. The third append compares size 2 with capacity 1, sees “not full”, and writes slot 2 of a two-slot block.

Growing by a constant. `self._capacity + self.growth` instead of `self._capacity * self.growth` passes every other test and fails one, `test_doubling_keeps_resizes_logarithmic`:

```
AssertionError: 500 reallocations for 1000 appends – are you growing by a constant instead  
assert 500 < 20
```

With `+ 2` the capacity goes 1, 3, 5, 7, ..., so there is a resize every second append. The array still *works* — every value is kept — which is exactly why the test counts resizes rather than checking values. Part 5 measures what this costs.

Forgetting `self.resize_count += 1`. The array works, so only `test_doubling_keeps_resizes_logarithmic` fails:

```
AssertionError: resize_count never went up – increment it in _resize
assert 0 > 0
```

The counter is not decoration: Part 5 and the notebook read it to show the doubling as a staircase. Look at the last line of `_resize`.

Part 2 — Reading and writing: `__getitem__` and `__setitem__`

2.1 The idea

Lecture 04, “Reading and writing are still $O(1)$ ”. Two rules, both about the difference between size and capacity:

- the valid indices are **0 to size – 1**. The slots from size to capacity – 1 exist in the Array, but reading one must raise `IndexError`, exactly as with a Python list;
- a negative index counts from the **last used** slot: add size, not capacity. The course Array has no negative indices, so your class provides them — in $O(1)$, by one addition.

The lecture prints `__getitem__` in full; it is five lines. Read it there, then write `__setitem__` yourself — the same translation and the same bounds check, then a write instead of a read.

2.2 Draw it

Draw a block of capacity 8 holding 10, 20, 30, 40, 50. Under each slot write its positive index; under the five used slots also write the negative index a user would type (–5 ... –1). Mark the slots a user may never reach.

```
a = DynamicArray([10, 20, 30, 40, 50])
show(a)
```

Checkpoint 2. For `a = DynamicArray([10, 20, 30, 40, 50])`, what does each line give — a value or an `IndexError`?

- (a) `a[-1]` (b) `a[-5]` (c) `a[5]` (d) `a[-6]`
 (e) `a._block[5]` (f) `a._block[-1]`

2.3 Write it

```
__setitem__(self, index, value):
```

1. if index is negative, add `self._size`;
2. if the result is not in `0 .. size – 1`, raise `IndexError`;
3. write value into `self._block[index]`.

Do not skip step 2 because “the Array will raise anyway”. It will not: slot 6 of an eight-slot block is perfectly valid to the Array, so `a[6] = 'x'` would silently write into a slot the user cannot see. `test_setitem_out_of_range_raises` checks exactly this: `arr[3] = 99` on `DynamicArray([1, 2, 3])` must raise.

2.4 Test it

```
pytest tests/test_dynamic_array.py -v -k "index or setitem"
```

Seven tests: `test_append_and_index`, `test_negative_indexing`, `test_index_out_of_range_raises` three times (for indices 3, -4 and 99 on a three-element array), `test_setitem` and `test_setitem_out_of_range_raises`.

2.5 When it fails

Checking against the capacity. `if not 0 <= index < self._capacity:`

```
FAILED tests/test_dynamic_array.py::test_index_out_of_range_raises[3]
E       Failed: DID NOT RAISE IndexError
```

`DynamicArray([1, 2, 3])` has capacity 4, so slot 3 exists and holds `None`; your method returned `None` instead of raising. Compare with `self._size`.

No negative translation. `test_negative_indexing` fails with

```
IndexError: Array index -1 out of range for length 4 (valid: 0..3; no negative indices)
```

The -1 went straight through to the course `Array`, which — like C — has no negative indices. Translate first.

No bounds check in `__setitem__`.

```
FAILED tests/test_dynamic_array.py::test_setitem_out_of_range_raises
E       Failed: DID NOT RAISE IndexError
```

The write went into slot 3 of the four-slot block: the `Array` saw nothing wrong, and `list(arr)` still shows `[1, 2, 3]` because slot 3 is past the size. Use the same check as `__getitem__`.

Adding the capacity. `index += self._capacity` fails two tests: `test_negative_indexing` with `IndexError: 3` (for `[1, 2, 3]`, -1 became 3, past the size) and `test_index_out_of_range_raises[-4]` with `DID NOT RAISE` (-4 became 0, a valid slot). -1 means the last **used** slot.

Part 3 — Inserting in the middle: `insert_at`

3.1 The idea

Lecture 04, “The contract with costs”. To insert at `index`, every element from `index` to `size - 1` moves one place right, then the new value goes into the gap. That is Week 2’s `insert_at` with two differences:

- Week 2 raised `OverflowError` when the array was full. Here a full array simply **resizes first** — that is the point of a dynamic array;
- Week 2 returned the new size. Here the size is an attribute: update `self._size`.

The index rule has not changed: `0 <= index <= size`, or `IndexError`.

Cost: $O(\text{size} - \text{index})$ shifts, so $O(n)$ at the front and $O(1)$ at the end.

3.2 Draw it

Draw `[1, 2, 3, 4]` (capacity 4, full) and `insert_at(0, 0)`. First the resize (capacity 8), then the shifts — draw each one as an arrow, numbered in the order you do them. Which element must move first?

Checkpoint 3. A student shifts from the **front**:

```
for i in range(index, self._size):
    self._block[i + 1] = self._block[i]
```

- What is `list(arr)` after `arr = DynamicArray([1, 2, 3])` and `arr.insert_at(1, 99)`?
- And after `arr = DynamicArray([1, 2, 3, 4])` and `arr.insert_at(0, 0)`?

3.3 Write it

- Check the index **first**, before anything changes: raise `IndexError` unless `0 <= index <= len(self)`. The docstring says so. `index == len(self)` is allowed and appends; a negative index is **not** translated here. That is the rule of Week 2's `insert_at` and of the linked list in Week 5. (Python's own `list.insert` quietly clamps a bad index instead. This course's structures do not.)
- If the array is full, resize (by the growth factor, as in `append`).
- Shift slots `index .. size - 1` one place right, **starting from the end**: slot `size` gets slot `size - 1`, then slot `size - 1` gets slot `size - 2`, and so on down to `index + 1`. `range(self._size, index, -1)` walks exactly those destinations.
- Write value into slot `index`; add one to `size`.

3.4 Test it

```
pytest tests/test_dynamic_array.py -v -k insert
```

Five tests:

- `test_insert_at_shifts_right`: `[1, 2, 3]` with 99 inserted at 1;
- `test_insert_at_out_of_range_raises`, three times: indices `-1`, `6` and `7` on `DynamicArray([1, 2, 3, 4, 5])` must raise `IndexError`. That array has 5 values in 8 slots, so slots `6` and `7` **exist** in the block: only your own check can reject them;
- `test_insert_at_grows_when_full`: 100 inserts at the front of an empty array. It resizes seven times on the way, and the result must be `[99, 98, ..., 0]`.

Then try the edges by hand, watching with `show(arr)`:

```
arr = DynamicArray([1, 2, 3, 4])      # full: capacity 4
arr.insert_at(0, 0)                  # must resize first
arr.insert_at(len(arr), 5)           # at the end: no shifts
print(arr, arr._capacity)            # DynamicArray([0, 1, 2, 3, 4, 5]) 8
```


3.5 When it fails

Shifting from the front — Checkpoint 3:

```
assert [1, 99, 2, 2] == [1, 99, 2, 3]
```

The 3 was overwritten by the 2 before it was moved. Walk from the end. `test_insert_at_grows_when_full` fails too, with `assert [99, 98, 98, 98, 98, 98, ...] == [99, 98, 97, 96, 95, 94, ...]`.

Forgetting to update the size.

```
assert [1, 99, 2] == [1, 99, 2, 3]
```

Everything moved correctly, but `size` still says 3, so `__iter__` stops one early.

Forgetting to resize when full. `test_insert_at_shifts_right` still passes: `DynamicArray([1, 2, 3])` has capacity 4, so it is not full. But `test_insert_at_grows_when_full` fails on its second insert:

```
IndexError: Array index 1 out of range for length 1 (valid: 0..0; no negative indices)
```

The first shift wrote slot 1 of the one-slot block the array started with. Resize first, exactly as in `append`.

Clamping the index like `list.insert`. All three out-of-range tests fail:

```
FAILED tests/test_dynamic_array.py::test_insert_at_out_of_range_raises[-1]
FAILED tests/test_dynamic_array.py::test_insert_at_out_of_range_raises[6]
FAILED tests/test_dynamic_array.py::test_insert_at_out_of_range_raises[7]
E      Failed: DID NOT RAISE IndexError
```

No index check at all. Two of them fail:

```
FAILED tests/test_dynamic_array.py::test_insert_at_out_of_range_raises[6]
FAILED tests/test_dynamic_array.py::test_insert_at_out_of_range_raises[7]
E      Failed: DID NOT RAISE IndexError
```

With spare room the course `Array` sees nothing wrong: `insert_at(7, 99)` returns quietly, the array still reads `[1, 2, 3, 4, 5, None]`, and the 99 is hidden in slot 7. Index `-1` does raise — but only from the `Array`, in the middle of the shifting loop, *after* the values have already moved: the array is left as `[1, 1, 2, 3, 4]`. Check the index yourself, **before** anything moves. A check placed after the loop fails `test_insert_at_out_of_range_raises[-1]` with `AssertionError: check the index BEFORE shifting anything`, showing that `[1, 1, 2, 3, 4]`.

Part 4 — Removing: `pop`

4.1 The idea

`pop(index=-1)` removes and returns the value at `index`. It is Week 2's `remove_at`, inside the class:

- the default `-1` means the last element — nothing shifts: **$O(1)$** ;
- any other index shifts slots `index + 1 .. size - 1` one place **left**: **$O(\text{size} - \text{index})$** .
`pop(0)` moves every other element.

That last line is the most important cost of the week. In Week 7, `SlowQueue` dequeues with `DynamicArray.pop(0)`, and you will measure what it costs (Lecture 04, "What it is built for").

4.2 Draw it

Draw `[10, 20, 30, 40]` (capacity 4). Then `pop(0)`: circle the value returned, draw the three left shifts as numbered arrows, and write what is left in slot 3.

Checkpoint 4. Starting from `arr = DynamicArray([10, 20, 30, 40])`, give the value returned, `Len(arr)` and `List(arr._block)` after each step:

- (a) `arr.pop()` (b) `arr.pop(0)` (c) `arr.pop(-1)`
(b) `arr.pop(5)` (e) `arr.pop()` (f) `arr.pop()`

4.3 Write it

1. If the array is empty, raise `IndexError` (the docstring and `test_pop_on_empty_raises` require it).
2. Translate a negative index by adding `size`; if the result is not in `0 .. size - 1`, raise `IndexError`.
3. Keep the value at `index` — you return it at the end.
4. Shift slots `index + 1 .. size - 1` one place left, **starting from the front**: slot `index` gets slot `index + 1`, and so on. (Left shifts go front-to-back; right shifts in Part 3 went back-to-front. Draw it if unsure.)
5. Subtract one from `size`, then set the slot that just fell out of use to `None`.
6. Return the value.

Why step 5's `None`? The old last slot still refers to an object. Nobody can reach it through your class any more, but the garbage collector can see the reference, so the object is never freed. Lecture 04's table says to "clear the freed slot", and `test_pop_clears_the_freed_slot` checks it. Question bank W4-B4 is this bug.

4.4 Test it

```
pytest tests/test_dynamic_array.py -v -k pop
```

Three tests: `test_pop_from_end_and_middle` (`pop()` then `pop(0)` on `[1, 2, 3]`), `test_pop_clears_the_freed_slot` (the same two pops on `['a', 'b', 'c']`, after which slots 1 and 2 of the block must be `None`) and `test_pop_on_empty_raises`. Then run the whole file — all 19 must pass:

```
pytest tests/test_dynamic_array.py -v
```

4.5 When it fails

No negative translation. `pop()` passes `-1` to the Array:

```
IndexError: Array index -1 out of range for length 4 (valid: 0..3; no negative indices)
```

in `test_pop_from_end_and_middle` and `test_pop_clears_the_freed_slot`. Same fix as Part 2 — the default argument *is* a negative index.

Forgetting to clear the freed slot. Only `test_pop_clears_the_freed_slot` fails:

```
AssertionError: assert 'b' is None
```

After `pop()` and `pop(0)` on `['a', 'b', 'c']`, the size is 1, but slot 1 still holds 'b': it was shifted into slot 0 and never cleared behind it.

Shifting the wrong way (`self._block[i + 1] = self._block[i]`):

```
assert [1] == [2]
```

The first element was copied rightwards over its neighbours instead of the neighbours moving left.

Passing by accident. Leave out the empty check and translate `-1` anyway: `DynamicArray().pop()` still raises — but it is the Array complaining, `Array index -1 out of range for length 1 (valid: 0..0; no negative indices)`, not your class. The test only asks for *an* `IndexError`, so it passes. Raise your own, with a message that tells the user what happened.

A bug no test catches. A shift loop that runs one step too far, `range(index, self._size)`, reads slot size. That works while there is spare capacity, and every test pops from an array that has some, so all 19 pass. On a full array it crashes: `DynamicArray([1, 2, 3, 4]).pop(0)` gives `IndexError: Array index 4 out of range for length 4 (valid: 0..3; no negative indices)`. Try it by hand with `show(arr)`.

Part 5 — The growth factor experiment

5.1 What the skeleton lets you change

`growth` is a constructor argument, so you can try other **factors** without touching your class:

```
for g in (2, 3, 10):
    arr = DynamicArray(growth=g)
    for i in range(1000):
        arr.append(i)
    print(g, arr.resize_count, arr._capacity)
```

It must be a whole number greater than 1. `make_block` builds an Array, and an Array needs a whole number of slots — so `DynamicArray(growth=1.5)` fails at its first resize with `TypeError: 'float' object cannot be interpreted as an integer`. (Lecture 04, “Any constant factor works”, shows the rounding you would need.)

Checkpoint 5. (a) Predict the three lines the loop above prints. (b) `DynamicArray([1], growth=1)` works. What does `DynamicArray([1, 2], growth=1)` do, and why?

5.2 Growing by a constant instead

The skeleton only multiplies. To grow by a constant **step**, write a subclass in your notebook that ignores the capacity it is asked for:

```
class StepArray(DynamicArray):
    """Grows by a constant 10 slots instead of by a factor."""

    def _resize(self, capacity):
        super()._resize(self._capacity + 10)
```

Your `append` still calls `_resize` when full; the subclass just changes the answer to “how big?”. Keep it out of `dsa/dynamic_array.py`, so the tests keep checking the real class (Homework 4, item 2).

Now count. To see the copying itself, not just the number of resizes, add a counter in the same way:

```
class Counting(DynamicArray):
    def __init__(self, values=(), growth=2):
        self.copies = 0  # before super().__init__, which appends
        super().__init__(values, growth)

    def _resize(self, capacity):
        self.copies += len(self)  # a resize copies every element
        super()._resize(capacity)

class CountingStep(Counting):
    def _resize(self, capacity):
        super()._resize(self._capacity + 10)

for n in (1000, 2000, 4000, 8000):
    d, s = Counting(range(n)), CountingStep(range(n))
    print(n, d.resize_count, d.copies, s.resize_count, s.copies)
```

You should get:

n	× 2: resizes	× 2: copies	+ 10: resizes	+ 10: copies
1,000	10	1,023	100	49,600
2,000	11	2,047	200	199,200

n	× 2: resizes	× 2: copies	+ 10: resizes	+ 10: copies
4,000	12	4,095	400	798,400
8,000	13	8,191	800	3,196,800

Read the columns as n doubles. With doubling, resizes go up by **one** (they are $\log_2 n$) and copies **double**, staying under $2n$ — linear. With a constant step, resizes double and copies go up **four times** — quadratic, roughly $n^2 / (2 \times 10)$. That is Lecture 04's "The three side by side", produced by your own class.

Part 6 — Measure it

Counting is exact; timing is what users feel. Both should tell the same story. Use `viz.complexity.measure` (it times the best of three runs and does not time building the input):

```
from viz.complexity import measure, plot_growth

def fill(arr_and_n):
    arr, n = arr_and_n
    for i in range(n):
        arr.append(i)

sizes = [500, 1000, 2000, 4000]
doubling = measure(fill, sizes, lambda n: (DynamicArray(), n))
step = measure(fill, sizes, lambda n: (StepArray(), n))
plot_growth({"x 2 (doubling)": doubling, "+ 10 (constant step)": step},
            reference=["n", "n^2"])
```

Expect the doubling curve to be a straight line close to the $O(n)$ reference, and the constant-step curve to bend upwards like $O(n^2)$. Your seconds will differ from your neighbour's; the **shapes** will not. Keep the sizes modest. Every element you move goes through the course Array's bounds check, so it is far slower than a Python list, and `measure` runs each size four times (one warm-up, then the best of three). Doubling n to 8,000 makes the constant-step run about four times longer, not twice.

Now the cost that sets up Week 7 — removing from the front against removing from the back:

```
def drain_front(arr):
    while len(arr):
        arr.pop(0)

def drain_back(arr):
    while len(arr):
        arr.pop()
```

```

sizes = [250, 500, 1000, 2000]      # small on purpose: see below
front = measure(drain_front, sizes, lambda n: DynamicArray(range(n)))
back = measure(drain_back, sizes, lambda n: DynamicArray(range(n)))
plot_growth({"pop(0) until empty": front, "pop() until empty": back},
            reference=["n", "n^2"])

```

This takes a few seconds, almost all of it in `drain_front`. Do not be tempted to add 8,000: emptying 8,000 elements with `pop(0)` moves 32 million elements, and four runs of that take minutes. That slowness is the result.

Checkpoint 6. Each time n doubles, by roughly what factor should each of these times grow: (a) filling a doubling array, (b) filling a `StepArray`, (c) `drain_front`, (d) `drain_back`?

Emptying an array with `pop(0)` shifts $n - 1$ elements, then $n - 2$, and so on: the triangular sum from Lecture 02, $\Theta(n^2)$ in total — $O(n)$ per removal. A queue that removes from the front of a dynamic array, one customer at a time, pays exactly that. In Week 7 you will build this `SlowQueue`, measure it, and then fix it with a ring buffer.

Part 7 — Exercises at a glance

Method	Target cost	The trap	Tests (-k)
<code>_resize(capacity)</code>	$O(\text{size})$	forgetting to switch <code>_block</code> , update <code>_capacity</code> or count <code>resize_count</code>	"constructor or grows_beyond or doubling" (3)
<code>append(value)</code>	amortised $O(1)$, worst $O(n)$	writing before the "is it full?" check; + instead of *	"constructor or grows_beyond or doubling" (3)
<code>__getitem__(index)</code>	$O(1)$	bounds against capacity; adding capacity to a negative index	"index or setitem" (7)
<code>__setitem__(index, value)</code>	$O(1)$	no bounds check — silently writes an unused slot	"index or setitem" (7)
<code>insert_at(index, value)</code>	$O(n)$	shifting from the front; no resize when full; clamping instead of raising <code>IndexError</code>	insert (5)

pop(index=-1)	O(1) at the end, O(n) elsewhere	no negative translation; not clearing the freed slot	pop (3)
---------------	---------------------------------	--	---------

Part 8 — Take-home practice (not graded)

1. **Question bank, Week 4, Part C** (docs/question-bank/week04-questions.md). Do W4-T1 and W4-T2 (doubling: which appends resize, total work, average per append) and W4-T3 (the same with a step of + 3), by hand, then check them with the Counting class of Part 5.
2. **Part E, W4-B1 to W4-B4**. Four buggy methods. You have now seen three of those bugs in your own code; say which.
3. **Part F, W4-C1 to W4-C3**, in practice/week04.py:

```
pytest tests/test_practice_week04.py -v
```

capacity_after and copies_for count without building anything. ShrinkingArray is a small dynamic array of your own that also halves when a pop leaves it a quarter full (Lecture 04, “Shrinking”). It is Homework 4’s item 4 in a form the tests can check.

4. **Homework 4, item 2** (Lecture 04, “This Week”): your Part 5 and Part 6 measurements, with resize_count plotted against n for both growth rules.

Answers to the question bank are in docs/question-bank/week04-answers.md, and the worked solutions in solutions/dsa/dynamic_array.py and solutions/practice/week04.py — for after you have tried.

Part 9 — Bridge to Lecture 05: linked lists

Everything expensive this week came from one fact: the elements sit **side by side**. That makes `a[i]` a single address calculation — and it makes `insert_at(0, x)` move every element, and growing copy them all.

Lecture 05 gives up contiguity. Each value lives in its own small **node**, anywhere in memory, holding a reference to the next one. Node is already given in `dsa/linked_list.py`; try it now:

```
>>> from dsa.linked_list import Node
>>> head = Node(3, Node(7, Node(1)))      # 3 -> 7 -> 1 -> None
>>> head = Node(0, head)                  # insert at the front
>>> head.value, head.next.value, head.next.next.next.value
(0, 3, 1)
```

Inserting at the front was one new node and one reference: nothing shifted, nothing was copied, however long the chain. Draw it:

```
from viz.draw import draw_linked_list
draw_linked_list([0, 3, 7, 1], highlight=0, title="after inserting 0 at the front")
```

The price is the other side of this week's table: to reach the fourth value you must follow three references, so indexing becomes $O(n)$. Lecture 04's last slide calls it "the opposite trade". Bring this week's cost table to the lecture and fill in the second column there.

Summary

Idea	The one line to keep
Size and capacity	Capacity is how many slots the Array has; size is how many hold data. $0 \leq \text{size} \leq \text{capacity}$.
Growing	Allocate a bigger Array, copy size elements, switch — $O(n)$, and count it.
append	Resize before writing; grow by $\text{capacity} * \text{growth}$.
Factor vs constant	$\times 2$: fewer than $2n$ copies for n appends. $+ c$: about $n^2/2c$.
Amortised $O(1)$	Any n appends cost fewer than $3n$; one append can still cost $O(n)$.
Indexing	Bounds are $0 \dots \text{size} - 1$; a negative index adds size, not capacity.
insert_at	IndexError unless $0 \leq \text{index} \leq \text{size}$; resize if full; shift right from the end . $O(n)$.
pop	Shift left from the front , clear the freed slot. $O(1)$ at the end, $O(n)$ at the front.
pop(0)	$O(n)$ — which is why Week 7's SlowQueue is slow.

Answers to the checkpoints

Checkpoint 1. `len(arr)` is 5, `arr._capacity` is 8, `arr.resize_count` is 3, and `list(arr._block)` is `['a', 'b', 'c', 'd', 'e', None, None, None]`. The resizes happened on the 2nd, 3rd and 5th appends (the array was full at sizes 1, 2 and 4), copying $1 + 2 + 4 = 7$ elements. The first append needed no resize: a new array starts with capacity 1.

Checkpoint 2. (a) $50 - -1 + 5 = 4$. (b) $10 - -5 + 5 = 0$. (c) IndexError: 5 — slot 5 exists in the Array but is past the size. (d) IndexError: $-1 - -6 + 5 = -1$, still out of range. (With the lecture's `raise IndexError(index)` the message shows the translated index.) (e) None — the Array itself has eight slots; this one is unused. (f) IndexError: Array index -1 out of

range for length 8 (valid: 0..7; no negative indices) — the course Array has no negative indices; only your class translates them.

Checkpoint 3. (a) [1, 99, 2, 2]. Slot 2 is copied into slot 3 *after* slot 1 has already overwritten slot 2, so the 3 is lost and the 2 is duplicated. (b) [0, 1, 1, 1, 1] — the same mistake repeated all the way along: the first element is smeared over every slot to its right. (The resize to capacity 8 still happens first, so there is no IndexError.)

Checkpoint 4.

Step	Returns	len	list(arr._block)
start		4	[10, 20, 30, 40]
(a) pop()	40	3	[10, 20, 30, None]
(b) pop(0)	10	2	[20, 30, None, None]
(c) pop(-1)	30	1	[20, None, None, None]
(d) pop(5)	IndexError	1	unchanged
(e) pop()	20	0	[None, None, None, None]
(f) pop()	IndexError — empty	0	unchanged

The capacity stays 4 throughout: this class never shrinks (Lecture 04, “Shrinking” — an extension, not part of the exercise). Only (b) shifted anything.

Checkpoint 5. (a)

```
2 10 1024
3 7 2187
10 3 1000
```

With factor r the capacity after the resizes is 1, r , r^2 , ..., and a resize happens each time the size reaches one of them, until the capacity reaches 1,000. For $r = 10$ the capacity is exactly 1,000 after three resizes, so the 1,000th append fits. (b) It raises `IndexError: Array index 1 out of range for length 1 (valid: 0..0; no negative indices)`. The first value fits in the one slot. The second finds the array full and calls `_resize(1 * 1)`, which copies into a new block that is *no bigger*; the write to slot 1 then fails. A growth factor must be greater than 1.

Checkpoint 6. (a) About $2\times$ — doubling is linear in total. (b) About $4\times$ — a constant step is quadratic: 798,400 copies become 3,196,800. (c) About $4\times$ — draining with `pop(0)` is quadratic, $\Theta(n^2)$ in total. (d) About $2\times$ — each `pop()` is $O(1)$, so n of them are linear. Timings are noisy, and at small n the constant-step curve can jump by more than $4\times$ (the quadratic part is still overtaking fixed costs). If you see $1.7\times$ or $5\times$, that is normal. Compare with the count table in Part 5, which has no noise at all.